

Task 1: Python List Memory Allocation

1. Introduction

A Python list is a dynamic array that stores references to objects. Unlike a fixed-size array in languages such as C++, a Python list can automatically grow or shrink during execution.

For example:

```
numbers = [10, 20, 30]
numbers.append(40)
```

The list automatically increases its storage capacity when necessary.

This flexibility makes Python lists convenient and efficient for general-purpose programming, but it requires Python to manage memory dynamically.

2. List Size vs List Capacity

Two concepts are important when discussing Python list memory:

List Size

The **size** is the number of elements currently stored in the list.

```
numbers = [10, 20, 30]
len(numbers)
```

Output:

3

Therefore:

List size = 3 elements

List Capacity

The **capacity** is the amount of space currently allocated internally for storing element references.

A list may have a capacity larger than its current size.

For example, conceptually:

```
Size:      3
Capacity:  4
```

The extra space allows Python to add another element without immediately allocating a completely new memory block.

Important: Python does not expose list capacity directly through the normal `list` API. It can be investigated indirectly using tools such as `sys.getsizeof()` or CPython internals.

3. Why Does Python Allocate Extra Space?

Suppose Python allocated exactly enough memory every time an element was added:

```
numbers = []
numbers.append(1)
numbers.append(2)
numbers.append(3)
numbers.append(4)
```

Python might need to repeatedly:

1. Allocate new memory.
2. Copy the existing references.
3. Release the old memory.

Doing this for every `append` would be expensive.

Instead, Python uses **overallocation**.

When a list needs more space, Python usually allocates additional unused capacity.

Conceptually:

```
Current size = 4
Current capacity = 4

append(5)

New size = 5
New capacity > 5
```

This gives the list room for future additions.

4. Python's Overallocation Strategy

CPython uses an overallocation strategy when resizing lists.

The exact resizing formula has changed between CPython versions, so it is better to understand the principle rather than memorize one fixed formula.

Modern CPython calculates a new allocated capacity using a formula based on the requested size and provides additional growth space.

Historically, the strategy has been approximately proportional to:

```
new capacity  $\approx$  old capacity + old capacity / 8
```

plus a small constant adjustment.

The important idea is:

Python does not normally increase a list's capacity by only one element. It reserves extra space to reduce the number of future reallocations.

For example, the capacity may grow in a pattern roughly like:

```
0
4
8
16
24
32
...
```

The exact values depend on the CPython version and resizing implementation.

5. What Happens When Capacity Is Exceeded?

Consider:

```
numbers = [1, 2, 3, 4]
```

Suppose, conceptually:

```
Size = 4
Capacity = 4
```

Now:

```
numbers.append(5)
```

There is no longer enough allocated space.

Python must resize the list.

The general process is:

1. Request more memory
- ↓
2. Allocate a larger memory area
- ↓
3. Copy/move the existing element references
- ↓
4. Add the new reference
- ↓
5. Release the old allocation if necessary

Conceptually:

Before:

```
[ A ][ B ][ C ][ D ]  
      capacity = 4
```

After resizing:

```
[ A ][ B ][ C ][ D ][ E ][   ][   ][   ]  
      larger capacity
```

The elements themselves are usually Python objects located elsewhere in memory. The list mainly stores **references/pointers to those objects**.

6. Memory Reallocation

When a list grows beyond its capacity, CPython needs to obtain a larger memory area.

Sometimes the underlying memory allocator can extend the existing allocation.

In other situations, a new memory block must be allocated.

If a new block is required:

Old memory:

```
[ref1][ref2][ref3][ref4]
```

↓ copy references

New memory:

```
[ref1][ref2][ref3][ref4][ref5][empty][empty]
```

The old allocation can then be released.

This process is called **reallocation**.

7. Time Complexity

Python list operations have different time complexities.

Operation	Average Complexity
Access by index	$O(1)$
Append	$O(1)$ amortized
Pop from end	$O(1)$
Insert at beginning	$O(n)$
Insert in middle	$O(n)$
Delete from beginning	$O(n)$
Search	$O(n)$

Why is append $O(1)$ amortized?

Most calls to:

```
numbers.append(x)
```

do not require resizing.

They simply place another reference into already allocated space.

Occasionally, however, a resize operation takes $O(n)$ time because existing references may need to be copied.

Therefore:

Most append operations $\rightarrow O(1)$

Occasional resize $\rightarrow O(n)$

Overall amortized append $\rightarrow O(1)$

8. Practical Example

```
numbers = []

for i in range(10):
    numbers.append(i)

print(numbers)
```

Output:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Python does not necessarily allocate memory separately for every `append()`.

Instead, it grows the internal array in stages.

We can inspect approximate memory usage:

```
import sys

numbers = []

for i in range(10):
    numbers.append(i)
    print(len(numbers), sys.getsizeof(numbers))
```

You may notice that `sys.getsizeof()` sometimes remains unchanged after an `append`.

For example, conceptually:

Length	Memory
0	X
1	Y
2	Y
3	Y
4	Y
5	Z
6	Z
...	

This demonstrates that Python has allocated capacity beyond the current number of elements.

9. Advantages of Overallocation

1. Faster append operations

Most `append()` operations do not require memory reallocation.

2. Amortized $O(1)$ performance

Appending repeatedly is efficient even though occasional resizing is expensive.

3. Convenient dynamic behavior

Programmers do not need to manually specify the maximum list size.

4. Good general-purpose performance

Python lists are highly optimized for common operations such as:

```
append()  
pop()  
indexing  
iteration
```

10. Trade-offs

Overallocation also has disadvantages.

Extra memory usage

The list may reserve memory that is currently unused.

For example:

```
Actual elements: 100  
Allocated capacity: 120
```

Some memory is unused but reserved for future growth.

Reallocation cost

When the capacity is exceeded, resizing can temporarily require additional memory and copying.

Cache and memory overhead

Python lists contain references to Python objects rather than storing many values directly inside the list's array.

Therefore, a list of integers can use considerably more memory than a compact native integer array.

11. Conclusion

Python lists use a **dynamic array** design combined with **overallocation**.

The important distinction is:

```
Size      = number of elements currently stored
Capacity = number of elements that can be stored before resizing
```

When capacity is exceeded, Python increases the allocation and may copy the existing references.

This gives:

```
Fast append operations
+
Automatic resizing
+
Amortized  $O(1)$  append performance
```

at the cost of:

```
Extra memory
+
Occasional expensive resizing
```

Overall, this is an excellent trade-off for a general-purpose programming language.

Task 2: Deep Copy vs Shallow Copy vs Assignment

1. Introduction

Python variables do not directly contain objects in the same way that a simple variable in some languages might.

Instead, variables are **names that refer to objects**.

For example:

```
x = [1, 2, 3]
```

Conceptually:

x $\longrightarrow$ [1, 2, 3]

Understanding references is essential for understanding:

- Assignment
 - Shallow copying
 - Deep copying
-

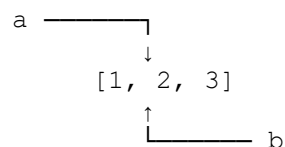
2. Assignment

Assignment does **not create a new object**.

Example:

```
a = [1, 2, 3]
b = a
```

Now:



Both variables refer to the same list.

Therefore:

```
b.append(4)
print(a)
```

Output:

```
[1, 2, 3, 4]
```

Changing `b` also changes what `a` sees because they reference the same object.

3. Identity Operator

Python provides the `is` operator to check whether two variables refer to the same object.

```
a = [1, 2, 3]
b = a

print(a is b)
```

Output:

```
True
```

This means:

```
a and b → same object
```

4. Shallow Copy

A **shallow copy** creates a new outer object but does not recursively copy objects inside it.

It can be created using:

```
copy.copy()
```

Example:

```
import copy

a = [1, 2, 3]
b = copy.copy(a)

print(a is b)
```

Output:

```
False
```

There are now two different list objects.

```
a → [1, 2, 3]
b → [1, 2, 3]
```

However, nested objects are still shared.

5. Nested Data Structures

Consider:

```
a = [[1, 2], [3, 4]]
```

Now:

```
import copy  
b = copy.copy(a)
```

The structure is approximately:

```
a → Outer List  
    ↓  
    [Inner List 1]  
    [Inner List 2]  
  
b → Different Outer List  
    ↓  
    [Same Inner List 1]  
    [Same Inner List 2]
```

Therefore:

```
b[0].append(99)  
print(a)
```

Output:

```
[[1, 2, 99], [3, 4]]
```

Even though `a` and `b` are different outer lists, they share the nested lists.

6. Deep Copy

A **deep copy** creates a new object and recursively copies the objects contained inside it.

Python provides:

```
copy.deepcopy()
```

Example:

```
import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)

b[0].append(99)

print(a)
print(b)
```

Output:

```
[[1, 2], [3, 4]]
[[1, 2, 99], [3, 4]]
```

The nested lists are independent.

Conceptually:

```
a → Outer List
    ↓
    Inner List A
    Inner List B

b → Different Outer List
    ↓
    Different Inner List A
    Different Inner List B
```

7. Comparison

Method	New Outer Object?	New Nested Objects?	Shared References?
Assignment	No	No	Yes
Shallow Copy	Yes	No	Nested objects shared
Deep Copy	Yes	Yes	Generally no

8. Three Examples Together

```
import copy

original = [[1, 2], [3, 4]]

assignment = original
shallow = copy.copy(original)
deep = copy.deepcopy(original)
```

We can test:

```
print(original is assignment)
print(original is shallow)
print(original is deep)
```

Output:

```
True
False
False
```

Now modify a nested list:

```
deep[0].append(100)
```

The original is unaffected.

But:

```
shallow[0].append(200)
```

also changes the nested list inside `original`.

9. Common Mistake

A common mistake is assuming:

```
new_list = old_list
```

creates a copy.

It does not.

Example:

```
numbers = [1, 2, 3]

new_numbers = numbers
```

```
new_numbers.append(4)

print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

Correct alternatives include:

```
new_numbers = numbers.copy()
```

or:

```
new_numbers = list(numbers)
```

or:

```
new_numbers = numbers[:]
```

These create shallow copies.

10. Performance and Memory Usage

Assignment

Assignment is extremely cheap.

```
b = a
```

Only another reference is created.

```
Time: O(1)
Memory: Very small
```

Shallow Copy

A shallow copy creates a new outer container.

For a list:

```
Time: O(n)
Memory: O(n)
```

because references in the outer list need to be copied.

Deep Copy

Deep copying can be much more expensive.

It recursively copies objects.

Depending on the structure:

Time: $O(n)$ or greater
Memory: $O(n)$ or greater

The exact cost depends on the number and complexity of objects.

11. When Should Each Be Used?

Assignment

Use assignment when you intentionally want multiple names to refer to the same object.

Example:

```
data = []  
backup_reference = data
```

Useful when shared modification is intended.

Shallow Copy

Use shallow copying when:

- You need a separate outer container.
- The nested objects can safely be shared.
- The structure is flat.
- You want better performance than deep copying.

Example:

```
original = [1, 2, 3]  
new = original.copy()
```

Deep Copy

Use deep copying when:

- You need a completely independent nested structure.
- Changes to nested objects must not affect the original.
- You are working with complex mutable data structures.

Example:

```
import copy

new_data = copy.deepcopy(original_data)
```

However, deep copy should not automatically be your default because it can consume significant memory and processing time.

12. Important Note About Immutable Objects

Immutable objects such as:

```
int
float
str
tuple
```

cannot be modified after creation.

For example:

```
x = 10
y = x
```

Changing `y`:

```
y = 20
```

does not modify `x`.

```
x → 10
y → 20
```


The behavior is different from modifying a mutable object such as a list.

13. Conclusion

The key difference is:

Assignment

↓

Same object

Shallow Copy

↓

New outer object

Shared nested objects

Deep Copy

↓

New outer object

New nested objects

Understanding this distinction prevents many bugs involving shared data.

Task 3: Memory Management in Python vs C++

1. Introduction

Memory management is the process of allocating, using, and releasing computer memory.

Python and C++ use significantly different approaches.

Python emphasizes:

Automatic memory management

+

Safety

+

Developer productivity

C++ provides:

Manual/control-oriented memory management

+
Deterministic lifetime
+
High performance

Neither approach is universally better. The appropriate choice depends on the application.

2. Stack and Heap Memory

Most programming languages distinguish conceptually between **stack** and **heap** storage.

Stack

Stack memory is commonly used for:

- Function call information
- Local variables
- Temporary data

Stack allocation and deallocation are generally very fast.

Example in C++:

```
void function()  
{  
    int x = 10;  
}
```

`x` is a local automatic variable whose lifetime is associated with the function scope.

Heap

Heap memory is used for dynamically allocated objects whose lifetime can extend beyond a particular function call.

C++ example:

```
int* p = new int(10);
```

The integer is dynamically allocated.

It must eventually be released:

```
delete p;
```

Modern C++ generally prefers smart pointers instead of raw `new/delete`.

3. Python Memory Management

Python manages memory automatically.

For example:

```
x = [1, 2, 3]
```

Python creates the necessary objects and manages their memory.

When an object is no longer needed, Python can reclaim its memory.

The exact details depend on the Python implementation, but **CPython**, the standard implementation most commonly used, primarily uses:

- Reference counting
 - A cyclic garbage collector
 - Specialized memory allocators
-

4. Reference Counting in Python

In CPython, objects maintain a reference count.

Consider:

```
a = [1, 2, 3]
```

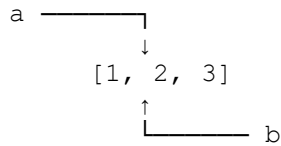
There is a reference to the list.

Then:

```
b = a
```

Now there are additional references to the same object.

Conceptually:



When references disappear:

```
del a
del b
```

the list may become unreachable.

When its reference count reaches zero in CPython, its memory can generally be reclaimed immediately.

5. Garbage Collection

Reference counting alone cannot handle every situation.

Consider a circular reference:

```
a = []
b = []

a.append(b)
b.append(a)
```

Now:



The objects reference each other.

Even if the program loses its external references to them, their reference counts may not reach zero.

CPython therefore also has a **cyclic garbage collector** that can detect certain unreachable reference cycles and reclaim them.

6. C++ Memory Management

C++ gives programmers much more direct control.

Dynamic allocation can be performed using:

```
int* p = new int(10);
```

and released using:

```
delete p;
```

For arrays:

```
int* numbers = new int[100];
```

```
delete[] numbers;
```

Incorrect memory management can cause serious problems.

7. Memory Leaks

A memory leak occurs when dynamically allocated memory is no longer needed but is not released.

Example:

```
void function()
{
    int* p = new int(10);
}
```

When the function ends, the pointer disappears, but the allocated integer may remain inaccessible.

This creates a memory leak.

8. RAII in C++

C++ provides an important design principle called:

RAII: Resource Acquisition Is Initialization

The idea is that an object's lifetime controls a resource's lifetime.

For example:

```
#include <fstream>

void function()
{
    std::ofstream file("data.txt");

    // Use file
}
```

When `file` goes out of scope, its destructor automatically closes the file.

RAII is not limited to files. It can manage:

- Memory
 - Locks
 - Sockets
 - Database connections
 - Other resources
-

9. Smart Pointers

Modern C++ provides smart pointers.

The main types are:

```
std::unique_ptr
std::shared_ptr
std::weak_ptr
```

unique_ptr

Represents exclusive ownership.

```
#include <memory>

std::unique_ptr<int> p = std::make_unique<int>(10);
```

When `p` goes out of scope, the object is automatically destroyed.

No manual:

```
delete
```

is required.

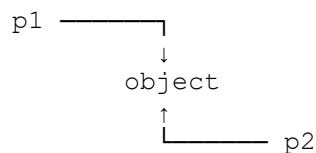
10. shared_ptr

`shared_ptr` allows multiple owners.

```
#include <memory>

auto p1 = std::make_shared<int>(10);
auto p2 = p1;
```

Conceptually:



The object remains alive while at least one owning `shared_ptr` exists.

11. weak_ptr

`weak_ptr` provides a non-owning reference to an object managed by `shared_ptr`.

It is particularly useful for avoiding certain ownership cycles.

Conceptually:

`shared_ptr` → Object ← `weak_ptr`

The `weak_ptr` does not keep the object alive.

12. Pointers and References

C++ provides explicit pointers:

```
int x = 10;
int* p = &x;
```

Here:

```
x = value  
p = address of x
```

A C++ reference provides another name for an existing object:

```
int x = 10;  
int& ref = x;
```

Python's model is different.

Python variables are names bound to objects:

```
x = [1, 2, 3]  
y = x
```

Both names refer to the same object.

Python does not expose raw memory pointers through ordinary application-level programming in the same way C++ does.

13. Object Lifetime

Python

Object lifetime is managed automatically.

An object generally remains alive while it is reachable.

Example:

```
x = [1, 2, 3]
```

Then:

```
x = None
```

The original list may become unreachable and eligible for memory reclamation.

C++

Object lifetime can be controlled by scope and ownership.

Example:

```
{  
    std::string name = "Alice";  
}
```

When the scope ends, the object's destructor runs.

This is deterministic.

14. Python vs C++ Comparison

Feature	Python	C++
Memory management	Mostly automatic	Manual + automatic tools
Garbage collection	Yes in CPython	No tracing GC by default
Reference counting	Yes in CPython	Used by <code>shared_ptr</code> , not general objects
RAII	Not the primary model	Fundamental
Raw pointers	Not normally exposed	Supported
Smart pointers	Not applicable in normal Python code	<code>unique_ptr</code> , <code>shared_ptr</code> , <code>weak_ptr</code>
Object lifetime	Mostly runtime-managed	Often deterministic
Manual <code>delete</code>	No	Possible with raw allocation
Memory control	Lower	Higher
Safety	Generally higher	Requires more care
Performance overhead	Runtime/object overhead	Can be very low-level and efficient

15. Memory Efficiency

Python objects generally carry significant runtime metadata.

For example:

```
x = 10
```

does not represent simply a four-byte integer in the way a typical C++ `int` might.

Python objects can contain information such as:

- Type information
- Reference-count information in CPython
- Object-specific data
- Alignment and allocator overhead

Therefore, large collections of small Python objects can consume substantial memory.

C++ can store primitive values compactly:

```
int numbers[1000000];
```

This can be much more memory-efficient than a Python list containing one million Python integer objects.

For numerical data, Python programs often use specialized libraries such as NumPy that store values in compact native arrays.

16. Performance

Python's automatic memory management improves developer productivity but introduces runtime overhead.

C++ can provide greater control over:

- Allocation
- Object layout
- Lifetime
- Data representation
- Memory locality

Therefore, C++ is often preferred for applications requiring extremely predictable performance.

Examples include:

- Game engines
- Operating systems
- Embedded systems
- Real-time applications
- High-performance computing

Python is often preferred for:

- AI and machine learning
 - Data analysis
 - Automation
 - Scripting
 - Rapid application development
-

17. Common Memory Problems

Python

Python prevents many manual-memory errors, but problems can still occur.

Excessive memory usage

```
data = []  
  
while True:  
    data.append("data")
```

The list continually grows.

Unwanted references

An object cannot be reclaimed if something still holds a reference to it.

Reference cycles

Objects can reference each other and require cyclic garbage collection.

C++

C++ can suffer from:

Memory leaks

Memory is allocated but never released.

Dangling pointers

A pointer refers to an object that has already been destroyed.

Example:

```
int* p = new int(10);  
  
delete p;  
  
// p is now dangling
```

Using `p` after deletion is unsafe.

Double deletion

```
delete p;  
delete p;
```

This is an error.

Buffer overflow

Writing beyond an allocated array can corrupt memory.

Use-after-free

Accessing memory after its lifetime has ended can produce undefined behavior.

18. Best Practices

Python

1. Avoid unnecessary copies.
 2. Use generators for large streams of data.
 3. Remove unnecessary references to large objects.
 4. Use appropriate data structures.
 5. Use specialized numerical arrays for large numerical datasets.
 6. Profile memory when working with large applications.
-

C++

1. Prefer RAII.
2. Prefer `std::unique_ptr` for exclusive ownership.
3. Use `std::shared_ptr` only when shared ownership is actually required.

4. Avoid unnecessary raw `new` and `delete`.
 5. Understand object lifetime.
 6. Avoid dangling references and pointers.
 7. Use standard containers such as `std::vector` and `std::string`.
-

19. Example: Python vs Modern C++

Python

```
def create_data():
    data = [1, 2, 3, 4, 5]
    return data

numbers = create_data()
```

Python automatically manages the list's memory.

C++

```
#include <memory>
#include <vector>

std::unique_ptr<std::vector<int>> create_data()
{
    return std::make_unique<std::vector<int>>(
        std::initializer_list<int>{1, 2, 3, 4, 5}
    );
}
```

Modern C++ uses RAII and smart pointers to make ownership explicit and automatic.

For this particular example, however, a value return would usually be simpler:

```
std::vector<int> create_data()
{
    return {1, 2, 3, 4, 5};
}
```

Modern C++ compilers can efficiently return such objects without requiring manual heap allocation.

20. Overall Comparison

The fundamental difference can be summarized as:

```
Python
  ↓
Automatic memory management
  ↓
Reference counting + garbage collection in CPython
  ↓
Easy and safe for most applications
```

while:

```
C++
  ↓
Explicit ownership + deterministic lifetimes
  ↓
RAII + smart pointers + manual control when needed
  ↓
Greater control and potentially lower overhead
```

21. Recommendations

Choose Python when:

- Rapid development is important.
- You are developing AI or ML applications.
- You are doing data analysis.
- You are writing automation scripts.
- Developer productivity is more important than low-level memory control.

Choose C++ when:

- Very high performance is required.
- Memory usage must be tightly controlled.
- Deterministic object lifetime is important.
- You are developing embedded or real-time systems.
- You are building game engines or performance-critical software.

Use both when appropriate

Many real-world systems combine the two.

For example:

Python



High-level application / AI



C/C++ libraries



Performance-critical operations



Hardware